MedPilot

OpenMRS Agentic Clinical Copilot Platform

Software Requirements Specification (SRS)

Document Version	1.0 — Initial Release
Classification	Confidential — Internal Use Only
Document Status	Draft for Review
Prepared By	MedPilot Product & Engineering Team
Date	April 2026
Standard	ISO/IEC 29148:2018 — Systems and Software Engineering
Regulatory Scope	HIPAA, HL7 FHIR R4, OpenMRS REST API v2

⚕ Intended Use Statement

MedPilot is a clinical workflow assistance platform. It is NOT a medical device, diagnostic tool, or autonomous prescribing system. All clinical decisions remain the responsibility of licensed healthcare professionals. MedPilot augments — but does not replace — clinical judgment.

© 2026 MedPilot. Proprietary and Confidential.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, non-functional, integration, and safety requirements for MedPilot — an agentic AI clinical copilot integrated with the OpenMRS electronic health record (EHR) platform. This document is intended for product managers, software engineers, clinical informatics specialists, QA engineers, and regulatory reviewers.

The document conforms to ISO/IEC 29148:2018 (Systems and Software Engineering — Requirements Engineering) and references HL7 FHIR R4 interoperability standards.

1.2 Scope

MedPilot enables clinicians, nurses, and healthcare administrators to interact with OpenMRS patient data and operational workflows using natural language. The system:

- Provides an intelligent query and response interface over structured EHR data
- Executes guarded write operations with explicit human confirmation
- Generates evidence-grounded clinical summaries and documentation drafts
- Delivers population-level analytics and operational reporting via conversational chat
- Maintains complete auditability of all user interactions and system actions

Scope Boundary

MedPilot operates exclusively within OpenMRS data boundaries. It does not connect to external clinical systems unless an optional integration module is explicitly configured and enabled.
The system does not perform autonomous clinical reasoning outside its defined tool scope.

1.3 Product Vision

MedPilot is:

"A clinical workflow copilot for OpenMRS that enables natural language interaction with patient data, operational analytics, and safe execution of clinical workflows."

MedPilot is NOT:

- A diagnostic AI or clinical decision support system (CDSS) with autonomous output
- An autonomous prescribing or medication management system
- A replacement for OpenMRS or any clinical EHR system
- A general-purpose chatbot without clinical data grounding

1.4 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
Agent	An AI orchestration unit that interprets user intent, selects tools, and coordinates responses
CDSS	Clinical Decision Support System — a system that provides clinicians with knowledge and patient-specific data to enhance decision-making
FHIR	Fast Healthcare Interoperability Resources — HL7 standard for exchanging healthcare data
EHR	Electronic Health Record — a digital record of a patient's medical history

© 2026 MedPilot. Proprietary and Confidential.

Term / Acronym	Definition
HIPAA	Health Insurance Portability and Accountability Act — US federal law governing PHI privacy
LLM	Large Language Model — the AI model underpinning natural language understanding and generation
NLP	Natural Language Processing — computational processing of human language input
OpenMRS	Open Medical Record System — an open-source EHR platform used globally
PHI	Protected Health Information — individually identifiable health information protected under HIPAA
RBAC	Role-Based Access Control — authorization model restricting access based on user roles
SRS	Software Requirements Specification — this document
Tool	A discrete, registered function callable by the agent to retrieve or modify data
Write Action	Any agent-initiated operation that creates or modifies data in OpenMRS

1.5 Document Conventions

Requirements are tagged with a priority classification:

Tag	Meaning
MUST	Mandatory — system cannot be accepted without this
SHOULD	Strongly recommended — deviation requires documented rationale
MAY	Optional — implementation at team discretion

1.6 Intended Audience

- Product Management — feature scope and acceptance criteria
- Software Engineering — implementation and API design
- Clinical Informatics — clinical safety and data accuracy requirements
- QA Engineering — testing and validation strategy
- Security & Compliance — HIPAA and audit requirements
- External Reviewers — regulatory, procurement, or partnership evaluation

2. Product Overview

2.1 Product Definition

MedPilot is an agentic AI platform integrated with OpenMRS, providing a conversational interface that enables healthcare staff to interact with patient and operational data using natural language. The system functions as a clinical workflow assistant — not a diagnostic AI or autonomous clinical actor.

2.2 Product Goals

#	Goal	Success Metric
G-01	Reduce chart navigation time for routine lookups	≥ 40% reduction in time-to-information vs. manual navigation
G-02	Reduce documentation burden for clinical staff	≥ 30% reduction in time spent on structured note entry
G-03	Improve longitudinal patient understanding	Clinician-rated improvement in summary utility ≥ 4.0/5.0
G-04	Enable natural language clinical workflows	All core query intents resolved with ≥ 90% accuracy
G-05	Provide operational analytics via chat interface	Population and operational queries resolved in < 3 seconds
G-06	Ensure safe, human-confirmed write actions	Zero unauthorized writes in testing and production
G-07	Maintain complete auditability of all actions	100% of agent actions captured in tamper-evident audit log

2.3 Explicit Non-Goals

⚠ **What MedPilot Must NOT Do**

- Autonomously diagnose patients or generate diagnostic conclusions
- Prescribe, recommend, or modify medications without explicit clinician confirmation
- Create, update, or delete any EHR record without a user-confirmed write action
- Replace the clinical judgment or legal responsibility of licensed practitioners
- Operate with or reference data outside the connected OpenMRS instance
- Fabricate, hallucinate, or infer clinical data not present in the source system

2.4 User Personas

Persona	Role	Primary Use Cases	Key Constraints
Dr. Amara	Attending Physician	Patient summaries, trend queries, note drafts	Limited time; requires evidence for every claim
Nurse Joy	Registered Nurse	Medication review, task creation, follow-up management	Reads frequently; writes with supervision
Admin Kwame	Clinic Administrator	Daily workload reports, population analytics, scheduling	No patient-level write access required
Dr. Priya	Specialist (Read-Only)	Cross-patient condition queries, lab trends	Strict read-only; no write capability

3. System Architecture

3.1 Architecture Overview

© 2026 MedPilot. Proprietary and Confidential.

MedPilot uses a layered agentic architecture in which user intent flows through a structured pipeline — from natural language input to data retrieval, reasoning, and response generation. All layers are independently deployable and observable.

3.2 Component Inventory

Component	Technology	Responsibility
Frontend UI	React 18+ or OpenMRS O3 Module	Chat panel, evidence viewer, timeline panel, action confirmation overlay
Backend API	FastAPI (Python) or Node.js (Express)	Session management, request routing, authentication enforcement
Agent Engine	LLM orchestration (LangChain / custom)	Intent classification, tool planning, response generation
Tool Layer	OpenMRS FHIR & REST adapters	Structured tool execution, data normalization, write confirmation flow
Data Layer	PostgreSQL + Redis	Audit log persistence, capability cache, session state
Model Gateway	LLM abstraction (provider-agnostic)	Token management, model routing, retry and fallback logic
Observability	Prometheus + Grafana or equivalent	Latency, token usage, error rates, tool call frequency

3.3 Agent Pipeline

The agent processes every user request through the following deterministic pipeline:

Step	Stage	Description
1	**Intent Detection**	NLP classification of user prompt into a defined intent type (see Section 8.1)
2	**Patient Resolution**	Identification and binding of the active patient context from the session or prompt
3	**Tool Planning**	Selection and sequencing of required tools based on classified intent and patient context
4	**Permission Check**	RBAC validation of each planned tool against the authenticated user's role
5	**Data Retrieval**	Parallel or sequential execution of selected read tools against OpenMRS APIs
6	**Reasoning Layer**	LLM synthesis of retrieved data into a coherent, evidence-linked response
7	**Evidence Building**	Attachment of source references (resource type, ID, timestamp, value) to each claim
8	**Confirmation Gate**	For write actions: generation and display of a structured preview requiring explicit confirmation
9	**Action Execution**	Execution of confirmed write actions via the Tool Layer and OpenMRS APIs
10	**Audit Logging**	Tamper-evident recording of all pipeline events, tool calls, data accessed, and actions taken

4. OpenMRS Integration Requirements

© 2026 MedPilot. Proprietary and Confidential.

4.1 FHIR R4 Resources

The system MUST support the following HL7 FHIR R4 resources via the OpenMRS FHIR module endpoint (/ws/fhir2/R4):

FHIR Resource	Clinical Use	Operations Required
Patient	Core identity, demographics, identifiers	read, search
Encounter	Visit records, encounter context, timing	read, search, create
Observation	Labs, vitals, clinical measurements, findings	read, search, create, update
Condition	Active and historical diagnoses	read, search, create
AllergyIntolerance	Documented allergies and adverse reactions	read, search, create
MedicationRequest	Prescription orders, active medication list	read, search
MedicationDispense	Dispensed medication records	read, search, create
ServiceRequest	Clinical orders, referrals	read, search
Task	Follow-up tasks, workflow actions	read, search, create, update

4.2 OpenMRS REST API Resources

The system MUST support the following OpenMRS REST API v1 resources as fallback and supplemental data sources:

REST Resource	Purpose
`/patient`	Patient search, demographics, identifier management
`/encounter`	Encounter listing, encounter type resolution
`/obs`	Observation retrieval where FHIR Observation is unavailable
`/condition`	Condition data fallback
`/allergy`	Allergy data fallback
`/order`	Clinical order management
`/drug`	Drug concept lookup and validation
`/location`	Facility and clinic location metadata
`/provider`	Clinician and care team metadata
`/concept`	Terminology and code system resolution

4.3 Capability Detection

At system startup, MedPilot MUST perform capability detection against the connected OpenMRS instance to enable adaptive behavior:

- Call /ws/fhir2/R4/metadata to retrieve the FHIR CapabilityStatement
- Parse and cache: supported resources, supported operations, and server feature flags
- Dynamically enable or disable tools based on detected capabilities
- Refresh capability cache on a configurable interval (default: 1 hour) or on connection error
- Log capability detection results to the observability layer

© 2026 MedPilot. Proprietary and Confidential.

4.4 Connector Abstraction Layer

The integration layer MUST expose a unified adapter interface to decouple agent tool logic from OpenMRS API implementation details:

```
// FHIRAdapter interface
interface FHIRAdapter {
  getPatient(patientId: string): Promise<Patient>
  getObservations(patientId: string, params?: ObsParams): Promise<Observation[]>
  getEncounters(patientId: string, params?: EncParams): Promise<Encounter[]>
  getConditions(patientId: string): Promise<Condition[]>
  getAllergies(patientId: string): Promise<AllergyIntolerance[]>
  getMedications(patientId: string): Promise<MedicationRequest[]>
  createObservation(obs: ObsPayload): Promise<Observation>
  createEncounter(enc: EncPayload): Promise<Encounter>
  createTask(task: TaskPayload): Promise<Task>
}

// RESTAdapter interface
interface RESTAdapter {
  getPatientREST(query: string): Promise<PatientResult[]>
  getConcept(conceptId: string): Promise<Concept>
  getDrug(drugId: string): Promise<Drug>
  createOrder(order: OrderPayload): Promise<Order>
  getMetadata(resourceType: string): Promise<Metadata>
}
```

5. Functional Requirements

5.1 Core Capability Domains

All functional requirements are organized into three capability domains, each with distinct intent types, data sources, and confirmation workflows.

5.1.1 Patient-Level Workflows

The system MUST support natural language queries and actions scoped to an individual patient context:

REQ-ID	Requirement	Priority	Category
FR-PAT-01	Retrieve and display comprehensive patient demographics (name, DOB, identifiers, contact)	MUST	Patient Data
FR-PAT-02	Generate a structured clinical summary including active conditions, medications, allergies, and recent encounters	MUST	Patient Summary
FR-PAT-03	Retrieve all observations for a specified concept (e.g., HbA1c, Blood Pressure) with timestamps and trend analysis	MUST	Observations
FR-PAT-04	Retrieve active medication list including dose, frequency, prescriber, and start date	MUST	Medications
FR-PAT-05	Retrieve and display documented allergies with reaction type and severity	MUST	Allergies
FR-PAT-06	Retrieve encounter history with encounter type, date, provider, and location	MUST	Encounters

© 2026 MedPilot. Proprietary and Confidential.

REQ-ID	Requirement	Priority	Category
FR-PAT-07	Display a longitudinal patient timeline combining encounters, observations, and conditions	MUST	Timeline
FR-PAT-08	Draft a clinical note based on retrieved patient data, formatted per configurable templates	SHOULD	Documentation
FR-PAT-09	Create a follow-up task with assignee, priority, due date, and clinical context — with confirmation	MUST	Tasks
FR-PAT-10	Add a structured observation (e.g., BP, glucose) to the patient record — with confirmation	MUST	Write Actions

5.1.2 Population Queries

The system MUST support aggregate queries across the full patient population without requiring individual patient context:

REQ-ID	Requirement	Priority	Category
FR-POP-01	Return total count of active patients registered in the system	MUST	Population
FR-POP-02	Return total encounter count for a specified date, date range, or relative period (e.g., 'today', 'this week')	MUST	Encounters
FR-POP-03	Return patient count filtered by a specified condition or diagnosis code	MUST	Population
FR-POP-04	Return patient count and list filtered by facility location	MUST	Population
FR-POP-05	Return encounter count and patient list filtered by attending provider	SHOULD	Population
FR-POP-06	Support multi-dimensional queries combining condition, location, date, and provider filters	SHOULD	Population

5.1.3 Operational Workflows

The system MUST support operational analytics and workflow management tasks:

REQ-ID	Requirement	Priority	Category
FR-OPS-01	Retrieve list of patients with encounters on a specified date, sorted by encounter time	MUST	Operational
FR-OPS-02	Identify patients flagged for follow-up based on configurable clinical criteria	MUST	Operational
FR-OPS-03	Generate a daily workload summary report including encounter count, provider distribution, and outstanding tasks	SHOULD	Reporting
FR-OPS-04	Bulk-create follow-up tasks for a filtered patient list — with confirmation per batch	SHOULD	Write Actions
FR-OPS-05	List all open tasks assigned to the authenticated user or their team	MUST	Tasks

6. Tool Framework

6.1 Tool Registry Specification

Every tool registered in the MedPilot Tool Registry MUST conform to the following schema:

```
interface Tool {
  name: string;               // Unique identifier, camelCase
  description: string;        // LLM-readable natural language description
  inputSchema: JSONSchema;    // Validated input parameter specification
  outputSchema: JSONSchema;   // Normalized output structure
  permissions: string[];      // Required RBAC permission strings
  isWrite: boolean;           // true = requires confirmation before execution
  timeout: number;            // Maximum execution time in milliseconds
  retryPolicy: RetryConfig;   // Retry behavior on transient failures
}
```

6.2 Required Read Tools

Tool Name	Description	Permission	Timeout
`searchPatients`	Full-text and identifier-based patient search	read:patient	3000ms
`resolvePatient`	Resolve patient from name, ID, or contextual reference	read:patient	2000ms
`getPatientDemographics`	Retrieve structured demographic data	read:patient	2000ms
`getConditions`	Retrieve active and historical diagnoses	read:clinical	3000ms
`getAllergies`	Retrieve allergy and adverse reaction records	read:clinical	2000ms
`getEncounters`	Retrieve encounter history with filters	read:clinical	3000ms
`getObservations`	Retrieve observations by concept, date range, or type	read:clinical	3000ms
`getMedicationRequests`	Retrieve active medication orders	read:clinical	3000ms
`getMedicationDispense`	Retrieve dispensed medication records	read:clinical	3000ms
`getTasks`	Retrieve open and completed tasks for patient or user	read:tasks	2000ms
`getTimeline`	Retrieve merged chronological event timeline	read:clinical	5000ms
`countPatients`	Return aggregate patient count with filters	read:population	3000ms
`countEncounters`	Return aggregate encounter count with date/provider filters	read:population	3000ms
`countPatientsByCondition`	Return patient count filtered by condition code	read:population	3000ms
`countPatientsByLocation`	Return patient count filtered by facility location	read:population	3000ms

6.3 Required Write Tools

Write Tool Policy

All write tools MUST present a structured preview to the user and require explicit confirmation before execution. Write tools MUST NOT execute if the agent confidence score is below 0.85. All write actions MUST be logged to the audit store with full payload, user, and timestamp.

Tool Name	Description	Permission	Confirmation
`createEncounter`	Create a new clinical encounter record	write:encounter	Required
`createObservation`	Add a structured observation to a patient record	write:observation	Required
`updateObservation`	Correct or update an existing observation value	write:observation	Required
`createCondition`	Add a new condition/diagnosis to the patient record	write:condition	Required
`createAllergy`	Document a new allergy or adverse reaction	write:allergy	Required
`createMedicationDispense`	Record a medication dispensing event	write:medication	Required
`createTask`	Create a follow-up or workflow task	write:tasks	Required
`createNote`	Create a structured clinical note	write:notes	Required

7. Natural Language Processing Requirements

7.1 Supported Intent Types

The intent classifier MUST resolve user prompts to one of the following canonical intent types:

Intent	Description	Example Prompt
`search_patient`	Locate a patient by name, ID, or descriptor	*"Find patient Maria Santos"*
`patient_summary`	Generate comprehensive patient overview	*"Summarize this patient."*
`trend_query`	Retrieve longitudinal value trends for a concept	*"Show HbA1c trend for last 6 months"*
`population_query`	Aggregate counts or lists across the population	*"How many diabetes patients were seen this week?"*
`operational_query`	Retrieve workflow or operational status	*"Which patients need follow-up?"*
`draft_note`	Generate a structured clinical note draft	*"Draft a SOAP note for today's encounter"*
`create_task`	Create a follow-up task for a patient	*"Create a follow-up task for patient in 2 weeks"*
`create_encounter`	Initiate a new encounter record	*"Start a new encounter for this patient"*
`write_observation`	Add a clinical observation to the record	*"Add BP reading of 140/90"*
`data_import`	Import or reconcile external data (optional module)	*"Import labs from Health Gorilla"*

7.2 Query Processing Requirements

REQ-ID	Requirement	Priority	Category
NLP-01	Classifier MUST achieve ≥ 90% accuracy on the canonical intent test suite	MUST	Accuracy
NLP-02	System MUST handle temporal references (today, yesterday, last 3 months) and resolve to absolute dates	MUST	Parsing

© 2026 MedPilot. Proprietary and Confidential.

REQ-ID	Requirement	Priority	Category
NLP-03	System MUST resolve ambiguous patient references (e.g., 'the patient', 'him/her') using active session context	MUST	Context
NLP-04	System MUST request clarification rather than assume when patient identity is ambiguous and no session patient exists	MUST	Safety
NLP-05	System SHOULD support follow-up questions within a conversation thread, maintaining context across turns	SHOULD	Continuity
NLP-06	System MUST respond in the same language as the user prompt where supported language packs are available	SHOULD	i18n
NLP-07	System MUST detect and refuse prompts that attempt to override safety guardrails (prompt injection detection)	MUST	Security

8. Evidence and Citation Requirements

Every clinical claim generated by MedPilot MUST be traceable to a specific source record. Evidence grounding is a core safety and trust requirement — not an optional feature.

8.1 Evidence Object Specification

```
interface EvidenceSource {
  resourceType: FHIRResourceType;  // e.g., 'Observation', 'Condition'
  resourceId: string;              // FHIR resource UUID
  date: ISO8601DateTime;           // Record date/time
  value: string;                   // Human-readable value
  units?: string;                  // Clinical units where applicable
  conceptCode?: string;            // LOINC / SNOMED / CIEL code
  provider?: string;               // Recording provider name or ID
}

interface EvidenceBoundClaim {
  claim: string;                   // The generated assertion
  confidence: number;              // 0.0 – 1.0 agent confidence score
  sources: EvidenceSource[];       // One or more supporting records
}
```

8.2 Evidence Requirements

REQ-ID	Requirement	Priority	Category
EV-01	Every clinical statement in a patient summary MUST be linked to at least one EvidenceSource record	MUST	Grounding
EV-02	Evidence panel MUST display resource type, record ID, date, and value for each cited source	MUST	UI
EV-03	Agent MUST NOT generate clinical claims if no supporting data exists; it MUST state data unavailability	MUST	Safety
EV-04	Trend statements MUST cite a minimum of two data points with timestamps and values	MUST	Accuracy

© 2026 MedPilot. Proprietary and Confidential.

REQ-ID	Requirement	Priority	Category
EV-05	Confidence score MUST be computed and logged for every response; scores below 0.70 MUST display a low-confidence warning	MUST	Trust
EV-06	Evidence references MUST be clickable, linking to the source record in OpenMRS where URL access is available	SHOULD	UX

9. Write Action Safety Framework

9.1 Confirmation Workflow

All write actions MUST follow the following five-step confirmation workflow without exception:

Step	Stage	System Behavior
1	**Intent Recognition**	Agent classifies the user prompt as a write intent type
2	**Payload Construction**	Agent constructs a structured data payload from extracted entities and retrieved context
3	**Preview Generation**	System renders a structured, human-readable preview of the exact data to be written
4	**User Confirmation**	System awaits explicit user confirmation ("Confirm") before any write is attempted; 'Cancel' aborts cleanly
5	**Execution & Audit**	System executes the confirmed write, receives the API response, and logs the full action to the audit store

9.2 Preview Example

```
┌──────────────────────────────────────────────┐
│  ⚠  Pending Write Action — Please Confirm    │
├──────────────────────────────────────────────┤
│  Action:      Create Observation             │
│  Patient:     Maria Santos (ID: PAT-00412)   │
│  Concept:     Blood Pressure                 │
│  Value:       140/90 mmHg                    │
│  Date:        2026-04-04T14:32:00Z           │
│  Provider:    Dr. Amara Diallo               │
│  Encounter:   ENC-00987                      │
├──────────────────────────────────────────────┤
│  [  CONFIRM  ]          [  CANCEL  ]         │
└──────────────────────────────────────────────┘
```

9.3 Write Safety Requirements

REQ-ID	Requirement	Priority	Category
WS-01	System MUST NOT execute any write operation without an explicit user confirmation event	MUST	Safety
WS-02	System MUST NOT execute write actions if agent confidence score is below 0.85	MUST	Safety

© 2026 MedPilot. Proprietary and Confidential.

REQ-ID	Requirement	Priority	Category
WS-03	Write preview MUST display all fields that will be written, with no hidden or omitted parameters	MUST	Transparency
WS-04	User MUST be able to edit the draft payload before confirming	SHOULD	UX
WS-05	System MUST log all write attempts (confirmed and cancelled) to the audit store	MUST	Audit
WS-06	Write tools MUST NOT be accessible to users with read-only RBAC roles	MUST	Authorization
WS-07	System MUST handle write API failures gracefully, displaying an error and NOT retrying without user acknowledgment	MUST	Reliability

10. User Interface Requirements

10.1 Core UI Panels

Panel	Requirements
Chat Panel	Primary interaction surface. MUST display: message thread, response with evidence links, confidence indicators, typing state, and error states. Supports multi-turn context.
Evidence Panel	Displays EvidenceSources for the active response. MUST show: resource type, record ID, date, value. SHOULD link to OpenMRS record.
Timeline Panel	Longitudinal view of encounters, observations, and conditions. MUST support date filtering and concept filtering.
Action Preview Panel	Write confirmation overlay. MUST display full payload, confirmation and cancel controls. MUST block interaction until resolved.
Patient Context Bar	Persistent bar showing active patient name, ID, DOB, and active condition count. MUST be visible in all views.

10.2 Accessibility Requirements

REQ-ID	Requirement	Priority	Category
UI-A-01	UI MUST conform to WCAG 2.1 Level AA accessibility guidelines	MUST	Accessibility
UI-A-02	All interactive elements MUST be keyboard-navigable	MUST	Accessibility
UI-A-03	Color MUST NOT be the sole means of conveying information (e.g., write action status)	MUST	Accessibility
UI-A-04	All UI text MUST be available to screen readers via appropriate ARIA labels	MUST	Accessibility

11. Security and Compliance Requirements

11.1 Authentication

© 2026 MedPilot. Proprietary and Confidential.

REQ-ID	Requirement	Priority	Category
SEC-AUTH-01	System MUST authenticate all requests using the active OpenMRS session token	MUST	Authentication
SEC-AUTH-02	System MUST support integration with institutional SSO providers (SAML 2.0, OAuth 2.0 / OIDC)	SHOULD	Authentication
SEC-AUTH-03	Session tokens MUST be validated on every API request; expired tokens MUST result in immediate access denial	MUST	Authentication
SEC-AUTH-04	System MUST enforce session timeouts consistent with OpenMRS configuration	MUST	Authentication

11.2 Authorization (RBAC)

Permission	Clinician	Nurse	Administrator	Read-Only	
`read:patient`	✓	✓	✓	✓	
`read:clinical`	✓	✓	—	✓	
`read:population`	✓	✓	✓	✓	
`read:tasks`	✓	✓	✓	✓	
`write:encounter`	✓	Supervised	—	—	
`write:observation`	✓	Supervised	—	—	
`write:condition`	✓	—	—	—	
`write:medication`	✓	—	—	—	
`write:tasks`	✓	✓	✓	—	
`write:notes`	✓	✓	—	—	

11.3 Audit Logging

REQ-ID	Requirement	Priority	Category
SEC-AUD-01	System MUST log every agent interaction with: authenticated user ID, timestamp, patient ID (if applicable), full prompt text, intent classification, tools invoked, data resources accessed, and actions executed	MUST	Audit
SEC-AUD-02	Audit logs MUST be stored in a tamper-evident, append-only data store	MUST	Audit
SEC-AUD-03	Audit logs MUST be retained for a minimum of 7 years in compliance with HIPAA requirements	MUST	Compliance
SEC-AUD-04	Audit logs MUST be exportable in a structured format (JSON, CSV) for compliance review	MUST	Compliance
SEC-AUD-05	System MUST NOT log raw PHI in prompt text without encryption at rest	MUST	Privacy

© 2026 MedPilot. Proprietary and Confidential.

11.4 Data Privacy

REQ-ID	Requirement	Priority	Category
SEC-PRI-01	All data in transit MUST be encrypted using TLS 1.2 or higher	MUST	Encryption
SEC-PRI-02	All PHI at rest MUST be encrypted using AES-256 or equivalent	MUST	Encryption
SEC-PRI-03	LLM API calls MUST NOT transmit raw patient PHI to external providers without a BAA	MUST	Compliance
SEC-PRI-04	System MUST support deployment in a HIPAA-compliant cloud environment or on-premises	MUST	Deployment
SEC-PRI-05	PII/PHI de-identification MUST be applied before any model fine-tuning or telemetry export	MUST	Privacy

12. Non-Functional Requirements

12.1 Performance

Operation	Target SLA	Notes
Patient summary generation	< 5 seconds (P95)	Measured from query submission to full response render
Single-concept observation query	< 3 seconds (P95)	Single FHIR resource fetch + response
Population count query	< 3 seconds (P95)	Aggregation across full patient population
Write action preview generation	< 2 seconds (P95)	From user prompt to confirmation modal display
Write action execution	< 3 seconds (P95)	From confirmation to API response confirmation
Maximum request timeout	10 seconds	All requests MUST fail gracefully with user notification beyond this threshold

12.2 Reliability

REQ-ID	Requirement	Priority	Category
NFR-REL-01	System MUST implement exponential backoff retry logic for transient OpenMRS API failures (max 3 retries)	MUST	Reliability
NFR-REL-02	System MUST implement circuit breaker pattern for external API dependencies with configurable thresholds	MUST	Reliability
NFR-REL-03	System MUST degrade gracefully when optional integrations are unavailable, clearly communicating limitations to the user	MUST	Reliability
NFR-REL-04	System MUST cache FHIR capability statements and concept metadata with configurable TTL	MUST	Performance
NFR-REL-05	Target system uptime MUST be ≥ 99.5% excluding scheduled maintenance windows	SHOULD	Availability

12.3 Observability

© 2026 MedPilot. Proprietary and Confidential.

REQ-ID	Requirement	Priority	Category
NFR-OBS-01	System MUST emit structured request logs including: request ID, user ID, intent, tool calls, latency, and status	MUST	Logging
NFR-OBS-02	System MUST expose Prometheus-compatible latency and error rate metrics per endpoint and tool	MUST	Metrics
NFR-OBS-03	System MUST track LLM token consumption per request and aggregate by user, intent, and time period	MUST	Cost
NFR-OBS-04	System MUST emit tool-level call frequency, success rate, and latency metrics	MUST	Metrics
NFR-OBS-05	Dashboards SHOULD be provided for real-time monitoring of system health and model performance	SHOULD	Operations

13. Safety Guardrails

Clinical Safety Classification

MedPilot's safety guardrails are non-negotiable constraints. Any requirement in this section classified as MUST represents a patient safety or regulatory compliance requirement and MUST NOT be waived, deferred, or partially implemented without formal clinical safety review.

REQ-ID	Requirement	Priority	Category
SAF-01	System MUST NEVER generate or display clinical data not present in the OpenMRS data source	MUST	Hallucination Prevention
SAF-02	System MUST NEVER generate autonomous diagnostic conclusions or clinical recommendations	MUST	Clinical Boundary
SAF-03	System MUST NEVER execute a write action without explicit user confirmation	MUST	Write Safety
SAF-04	System MUST NEVER operate in a patient context if no patient is identified and confirmed	MUST	Context Integrity
SAF-05	System MUST respond 'I cannot find data for this request' when queried data is absent, rather than inferring or extrapolating	MUST	Data Integrity
SAF-06	System MUST detect and block prompt injection attempts that attempt to override safety rules	MUST	Security
SAF-07	System MUST display a low-confidence warning when agent confidence is below 0.70 on any clinical claim	MUST	Trust
SAF-08	System MUST NOT expose data from Patient A in a session contextualized to Patient B	MUST	Data Isolation
SAF-09	System MUST rate-limit write actions per user session to prevent inadvertent bulk data modification	SHOULD	Write Safety

14. Testing Requirements

© 2026 MedPilot. Proprietary and Confidential.

14.1 Test Coverage Requirements

Test Category	Scope	Acceptance Threshold
Unit Tests	Individual tool functions, input validation, output schema conformance	≥ 90% code coverage per tool
Integration Tests	FHIR API calls, REST API fallback behavior, capability detection	All defined resources successfully tested against live sandbox
Agent Intent Tests	Intent classification accuracy across the canonical prompt test suite	≥ 90% classification accuracy
Evidence Tests	All responses verified to include evidence citations matching source data	100% clinical claims have ≥ 1 source
Safety Tests	No write without confirmation; no hallucinated data; guardrail trigger tests	0 safety violations in test suite
Performance Tests	Latency under load for all SLA-bound operations	All P95 targets met under 50 concurrent users
Security Tests	RBAC enforcement, prompt injection detection, session isolation	0 unauthorized data access in test suite
Regression Tests	Prompt output stability across model and version updates	< 5% output variation on canonical prompt set
Accessibility Tests	WCAG 2.1 AA compliance across all UI panels	0 critical accessibility violations

15. Acceptance Criteria

The system MUST satisfy all of the following acceptance criteria before production release:

REQ-ID	Requirement	Priority	Category
AC-01	All clinical response claims are linked to a minimum of one EvidenceSource record	**MUST**	Evidence
AC-02	All write actions require and receive explicit user confirmation before execution	**MUST**	Write Safety
AC-03	Population and operational queries return accurate results within the defined SLA	**MUST**	Performance
AC-04	FHIR capability detection correctly identifies supported resources on OpenMRS startup	**MUST**	Integration
AC-05	REST API fallback activates correctly when FHIR resources are unavailable	**MUST**	Integration
AC-06	RBAC enforcement prevents unauthorized access to all write tools by restricted roles	**MUST**	Security
AC-07	Audit logs are complete for all agent interactions in the end-to-end acceptance test suite	**MUST**	Compliance
AC-08	Intent classifier achieves ≥ 90% accuracy on the canonical acceptance test prompt set	**MUST**	NLP
AC-09	All P95 performance SLAs are met under a 50 concurrent user load test	**MUST**	Performance

© 2026 MedPilot. Proprietary and Confidential.

REQ-ID	Requirement	Priority	Category
AC-10	Safety guardrail tests pass with 0 violations (no hallucination, no silent writes, no cross-patient data exposure)	MUST	Safety

16. Feature Roadmap

Phase	Name	Capabilities	Target
Phase 1	**Core Read & Query**	Patient-level queries, population analytics, evidence grounding, read-only workflows, audit logging	GA Release
Phase 2	**Clinical Write Actions**	Draft clinical notes, create tasks, create encounters, add observations — all with confirmation workflow	Q2 Post-GA
Phase 3	**Data Reconciliation**	External data import, medication reconciliation, allergy deduplication, data quality workflows	Q3 Post-GA
Phase 4	**External Integrations**	Health Gorilla, IMO terminology, FDB drug data — all modular and independently configurable	Q4 Post-GA
Phase 5	**Proactive Intelligence**	Configurable clinical alerts, care gap identification, protocol adherence monitoring, proactive task creation	Future

17. Optional Integrations

The following external integrations are optional modules that MUST be independently configurable, individually disableable, and isolated such that their unavailability does not impact core MedPilot functionality:

Integration	Purpose	Requirement
Health Gorilla	External lab and imaging results import	MUST support FHIR-compatible import; MUST require user confirmation for reconciliation
IMO (Intelligent Medical Objects)	Clinical terminology standardization and enrichment	SHOULD map local OpenMRS concepts to IMO codes; MUST degrade gracefully when unavailable
FDB (First Databank)	Drug interaction checking and medication data enrichment	SHOULD surface interaction alerts during medication write actions; MUST be opt-in per deployment

Appendix A: Example Supported Queries

Category	Example Prompt	Expected System Behavior
Patient	`"Summarize this patient."`	Returns structured summary: conditions, medications, allergies, recent encounters, with evidence citations
Patient	`"Show John's last HbA1c."`	Returns most recent HbA1c observation with date, value, units, and source record ID

© 2026 MedPilot. Proprietary and Confidential.

Category	Example Prompt	Expected System Behavior
Patient	*"What medications is this patient on?"*	Returns active MedicationRequest list with dose, frequency, and prescriber
Patient	*"Show allergy list."*	Returns AllergyIntolerance resources with reaction type and severity
Patient	*"Add note: patient reports dizziness."*	Drafts a structured observation, displays confirmation preview, awaits confirmation
Patient	*"Create a 2-week follow-up task."*	Constructs Task payload, displays preview with due date, awaits confirmation
Population	*"How many patients exist?"*	Calls countPatients(), returns total patient count with timestamp
Population	*"How many visits happened today?"*	Calls countEncounters(date=today), returns count
Population	*"How many diabetes patients were seen this week?"*	Calls countEncounters + countPatientsByCondition with condition filter, returns combined result
Operational	*"Show patients needing follow-up."*	Returns patient list with open follow-up tasks or overdue visit flags
Operational	*"List patients seen today."*	Returns encounter list for current date, patient names and encounter times
Operational	*"Generate daily workload report."*	Produces encounter count, provider distribution, open tasks summary

© 2026 MedPilot. Proprietary and Confidential.